

German Electricity Demand: Time Series Modelling Case Study

Forecasting German electricity load (2015–2020) using benchmark models, SARIMA/SARIMAX, a feature-based regression model, and an LSTM, with Berlin temperature as an exogenous variable.

Repository contents

.

- └─ German Electricity Load Forecasting.ipynb
- └─ README.md

How to run

Google Colab (recommended):

1. Upload [German Electricity Load Forecasting.ipynb](#) to [Google Colab](#) (File → Upload notebook).
2. Runtime → Change runtime type → GPU (speeds up the LSTM section; everything else runs fine on CPU).
3. Run the **Setup** cell first — it installs all dependencies ([statsmodels](#), [scikit-learn](#), [tensorflow](#), [requests](#), [pyarrow](#)) inside the Colab environment, no separate install step needed.
4. In **Part 1**, either mount Google Drive and point [RAW_CSV](#) at your copy of the data, or run the upload cell and choose [time_series_60min_singleindex.csv](#) from your computer when prompted.
5. Run every cell top to bottom — later sections depend on variables and models created earlier.

Locally (Jupyter):

```
python -m venv venv
```

```
source venv/bin/activate    # Windows: venv\Scripts\activate
```

```
pip install pandas numpy matplotlib statsmodels scikit-learn tensorflow requests pyarrow jupyter
```

```
jupyter notebook "German Electricity Load Forecasting.ipynb"
```

Data sources

Data	Source	Notes
Electricity load	Open Power System Data , 60-minute resolution file	Filtered to DE_load_actual_entsoe_transparency , 2015-01-01 through end of file (Oct 2020)
Temperature	Open-Meteo Archive API	Berlin (lat 52.52, lon 13.41), fetched live in Part 4, resampled to weekly means

Notebook structure

Section	Contents
Setup	Imports, shared metric functions (RMSE/MAE/MAPE), RESULTS dict used by every later section
Part 1	Data load and resampling (hourly/daily/weekly), initial plots, seasonal decomposition, ADF/KPSS stationarity tests, ACF/PACF
Part 2	Benchmark forecasts: Mean, Naive, Seasonal Naive, Drift (104-week / 2-year horizon)
Part 3	SARIMA — AIC grid search over $p \in [0,6]$, $d \in [0,2]$, $q \in [0,6]$, residual diagnostics, forecast with confidence intervals
Part 4	Berlin temperature fetch and SARIMAX with temperature as an exogenous regressor
Part 5	Random Forest / Gradient Boosting with lag and calendar features, leakage-safe recursive forecasting
Part 6	LSTM on hourly data, walk-forward evaluation

Section	Contents
Part 7	Analysis questions (answered in the notebook/report, not auto-generated)
Final comparison	Evaluation metrics table for all models + single overlay plot of every forecast vs. actual
Discussion	Critical analysis and future improvements (written independently, see report)

Methodology notes

- **Forecast horizon:** last 2 years of weekly data (104 weeks) held out as the test set for all weekly models. The LSTM additionally evaluates on the equivalent hourly test horizon.
- **Conditional vs. operational forecasts:** the SARIMAX model (Part 4) is conditioned on *observed* Berlin temperature during the test period rather than a temperature forecast, so it is a conditional/explanatory forecast rather than a true operational one.
- **Leakage avoidance:** lag and rolling features in the feature-based model (Part 5) are built recursively during forecasting, using only real history plus the model's own prior predictions — never the true future load.
- **SARIMA seasonal order:** fixed at $(1, 1, 1, 52)$ rather than grid-searched, since with ~197 training weeks (~3.8 annual cycles) a full seasonal grid search is not well identified statistically; only the non-seasonal (p, d, q) grid is searched exhaustively by AIC.

Requirements

`pandas`, `numpy`, `matplotlib`, `statsmodels`, `scikit-learn`, `tensorflow`, `requests`, `pyarrow` — all installed automatically by the notebook's Setup cell on Colab.